KNDB: Epistemic Typing as a Database-Layer Primitive

[Author 1 Name]*
[Institution]
[City], [Country]
[email]

[Author 2 Name]
[Institution]
[City], [Country]
[email]

[Author 3 Name]
[Institution]
[City], [Country]
[email]

[Author 4 Name]
[Institution]
[City], [Country]
[email]

ABSTRACT

Databases enforce data types but are blind to data origin. A verified measurement, a model's inference, and a value computed from other values are stored identically, and the distinction between them is lost at the moment of writing. This mattered less when humans wrote most records. It matters now: enrichment pipelines, reverse-ETL syncs, and AI agents write model-generated values into production stores beside verified facts at scale, while regulators increasingly require that automated decisions be explainable in terms of what they rested on. We argue that a fact's epistemic kind, whether it was measured, inferred, or derived, belongs at the database layer, enforced by triggers, constraints, and provenance extensions rather than in application code. We present KNDB, a prototype built on stock PostgreSQL and the ProvSQL extension, in which epistemic kind governs three things: what may be written (typed write rules checked by triggers before commit), which value survives a conflict (a precedence lattice in which a measurement cannot be displaced by an inference, regardless of arrival order or asserted confidence), and what a fact may be used for (compliance and training views that exclude inferred values by construction). Confidence composes through joins via semiring provenance, and bitemporal validity supports point-in-time reconstruction. On an adversarial suite of 70 trust-violating writes, KNDB rejects all 70; a plain schema rejects none and carefully written application-layer guards reject 30. A hand-built trigger schema also rejects all 70, which is precisely our point: the guarantee is achievable by hand, once, per application, forever. The alternative is to enforce it once in the engine, where no writer can forget it. We report costs honestly, including a size-dependent write penalty attributable to provenance bookkeeping, and outline the isolation regime under which its guarantees hold.

CCS CONCEPTS

• **Information systems** → **Data management systems**; *Integrity checking*; *Uncertainty in databases.*

KEYWORDS

epistemic typing, data provenance, write-time enforcement, precedence lattice, semiring provenance, bitemporal databases, PostgreSQL

1 INTRODUCTION

Relational databases enforce data types: a column declared numeric will not accept a string. They enforce nothing about data origin. A postal-verified address from a billing system, an unverified update typed by a sales rep, and a value computed from other rows are all just values of the declared type. Whatever distinguished them, that one was measured, one guessed, one computed, exists only in application code, column-naming conventions, or tribal knowledge, none of which the engine defends.

For decades this blindness was tolerable because most writers were humans or deterministic programs. That assumption no longer holds. Data-enrichment pipelines sync vendor-modeled attributes into CRM fields where verified values live; measured single-provider accuracy for such enrichment runs roughly 65–85%, and an analysis of twelve billion Salesforce records found 45% duplicate records overall but 80% among records created via API integrations. Machine-written data is demonstrably dirtier than human-entered data, and nothing in the schema says which is which [16]. Reverse-ETL products push model scores and predicted attributes into operational stores as a standard workflow. Recent work from Microsoft Research found that across nineteen frontier models, LLM agents on long delegated tasks silently degrade the content they write, with roughly 25% corruption over twenty interactions for the strongest models, and more degradation when the agents were given file-writing tools [11]. Meanwhile the consequences of conflated origin are documented: master-data platforms configured "most recent wins" let an unverified CRM address displace a postal-verified one [15]; the CFPB requires creditors to state accurate, specific reasons for adverse decisions even when produced by complex models [13]; Equifax was fined $15M in part for flawed code producing inaccurate scores [14]; and training pipelines must keep model-generated data out of verified corpora to avoid model collapse [10], today via entire external decontamination pipelines bolted onto stores that have no origin concept.

Our position is simple. The epistemic kind of a fact, whether it was measured, inferred, or derived, is a property of the data, and

*All four author slots are placeholders pending team sign-off. Do not submit this manuscript until every bracketed field ([Author N Name], [Institution], [City], [Country], [email]) is replaced with a real value.

it belongs at the database layer — enforced by triggers and constraints inside the DBMS rather than in application code: enforced at write time, carried through queries, and consulted at use time. Application code can implement these checks. But the argument for moving them into the engine is the one that moved referential integrity there decades ago. A database has many writers, those writers change over time, and a guarantee that depends on every one of them behaving is not a guarantee at all. Our evaluation makes this concrete.

This paper makes four contributions:

(1) **Epistemic typing as a database-layer primitive.** Every fact carries a kind (MEASURED, INFERRED, DERIVED) and a confidence; five write-time rules (§3) are enforced by BEFORE triggers, so an inferred value cannot enter a slot registered as measured, and a derived fact must cite its sources.
(2) **Epistemic precedence for conflict resolution.** Colliding writes are resolved by an ordered lattice — kind, then specificity, then confidence — under which a measurement cannot be silently displaced by an inference regardless of arrival order or asserted confidence. Losing facts are preserved in an audit relation with a machine-readable reason.
(3) **Kind-scoped use.** Views for compliance, analytics, and training-safe workloads make origin-scoped reading the default rather than a per-query convention, and a progressive-depth operator widens a query from measured-only through inferred and derived while reporting the confidence cost of the widening.
(4) **An honest evaluation.** On an adversarial suite, KNDB and a hand-built steelman both catch 70/70 violations while application-layer guards catch 30/70 and a plain schema 0/70; semiring-propagated confidence is exact where both baselines that skip it drift by 0.21 mean absolute error; and we quantify a size-dependent write cost attributable to provenance bookkeeping rather than claiming enforcement is free.

KNDB is deliberately not a new database. It is 508 lines of SQL and PL/pgSQL over stock PostgreSQL 17 and ProvSQL [2], reproducible from a fresh clone to a passing eleven-stage smoke suite in under ten seconds. Whether these mechanisms can be built is not the interesting question; anyone could build them this afternoon. The question is whether the community agrees they belong in the engine, as types and constraints do. This paper is our case that they do.

2 MOTIVATING FAILURES

Three documented patterns, one root cause.

Recency displacing verification. Master-data survivorship rules decide which source's value survives into the golden record. A practitioner who audits MDM deployments at financial institutions describes the most common configuration error: "most recent timestamp wins," under which a CRM's newer, rep-typed address displaces the billing system's postal-verified address [15]. The platform did exactly what it was told; the failure is that verification status was never a field the resolution logic could consult. The same audits find survivorship tuned to technical criteria such as most recent, most complete, or highest confidence, because origin is not represented.

Decisions that cannot explain themselves. US creditors must give specific, accurate reasons for adverse actions even when decisions are produced by complex models [13]. A decision computed by joining a verified bureau attribute with a model-estimated one is, in a standard store, just a flat result; nothing records that one input was measured and the other inferred, or how certain either was. The documented stakes are not hypothetical: credit-reporting complaints reached 5.8 million in 2025 (88% of all CFPB complaints), and Equifax's $15M penalty cited flawed code producing inaccurate scores [14].

Synthetic data leaking into training corpora. Training on model-generated data degrades models [10]; contamination persists even in widely used open datasets, and detection tooling exists precisely because the problem is routine [12]. Teams therefore run external decontamination pipelines (n-gram overlap checks, per-sample provenance chains) to reconstruct, outside the store, a distinction the store could have kept.

The root cause is identical in all three: the schema has no place to record how a value came to exist, so the distinction is lost at write time. This is not an error; it is the design. Everything downstream is reconstruction.

3 DESIGN

KNDB extends PostgreSQL through its standard extensibility surface: an ENUM and a DOMAIN, BEFORE triggers, range types with a GiST exclusion constraint, ProvSQL provenance tokens, and views. Enforcement lives entirely inside the engine; the Python in our repository is benchmark harness only.

3.1 Data model

Every fact is one row:

```
CREATE TABLE kndb.fact (
  fact_id uuid PRIMARY KEY,
  entity_id int NOT NULL,
  attribute text NOT NULL,
  value text NOT NULL,
  epistemic_kind kndb.epistemic_kind NOT NULL,
  confidence kndb.confidence NOT NULL,
  sources uuid[] NOT NULL DEFAULT '{}',
  specificity smallint NOT NULL DEFAULT 100,
  valid_time tstzrange NOT NULL,
  sys_time tstzrange NOT NULL,
  writer text NOT NULL DEFAULT current_user,
  EXCLUDE USING gist (entity_id WITH =,
    attribute WITH =, valid_time WITH &&)
  WHERE (upper(sys_time) = 'infinity'));
```

`epistemic_kind` is MEASURED (directly observed), INFERRED (output of a model or rule), or DERIVED (deterministic aggregate of other facts). `confidence` is a numeric domain on $[0, 1]$. `specificity` encodes write generality: 0 for batch defaults (nightly loaders, imputation jobs), 100 for normal per-entity writes, above 100 for adjudicated human corrections. The two ranges give bitemporality — valid time is when the fact held in the world, sys time when the store believed it — and the exclusion constraint guarantees at most one live fact per entity-attribute-interval.

3.2 Write-time typing rules

A BEFORE INSERT OR UPDATE trigger enforces five rules: (R1) a DERIVED fact must reference at least one source; (R2) every source must resolve to an existing fact; (R3) a MEASURED fact must have no upstream sources; (R4) an INFERRED fact must have confidence strictly below 1.0; (R5) if an attribute is registered in `kndb.slot_kind` as requiring kind K, the incoming row's kind must equal K. R5 is the rule that stops an imputation job writing into a measured slot: the write fails before commit, with an error naming the rule, rather than succeeding and being discovered in an audit months later.

3.3 Precedence lattice

When a new fact overlaps a live fact on the same entity and attribute with a different value, the conflict trigger resolves by an ordered comparison:

(1) **Kind**: MEASURED > DERIVED > INFERRED. An inferred write, at any confidence, cannot displace a live measured fact.
(2) **Specificity** (tie on kind): an entity-specific write beats a batch default; an adjudicated correction beats both.
(3) **Confidence** (tie on both): higher wins.
(4) **True tie**: the newer fact lands, recorded as `contradicted_same_rank`.

The losing fact — whether the incumbent or the newcomer — is written to `kndb_audit.evicted_fact` with the full original row and a machine-readable reason (`kind_outranked`, `specificity`, `confidence`, `contradicted_same_rank`). Nothing is silently erased; a steward can review why any value lost.

This directly repairs the survivorship failure of §2: under KNDB the postal-verified address is MEASURED, the CRM update is INFERRED or lower-specificity, and no timestamp ordering can invert that. Both last-writer-wins and confidence-wins are special cases the lattice deliberately rejects: the former ranks on time, the latter on a scalar a model asserts about itself, and a model is often most confident where it is most wrong.

When a source cited by a DERIVED fact's `sources[]` is later evicted by the lattice, the source row remains in the table with a closed `sys_time`; the derived row's references therefore still resolve under R2, and the derived fact remains live. This is a bitemporal choice, not an oversight: the DERIVED fact was validly derived from what the store believed at the time, and a bitemporal engine preserves that history rather than retroactively rewriting it. Flagging or recomputing derivations whose sources have moved on is a deliberate future-work extension, orthogonal to the write-time rules the paper claims.

3.4 Confidence through queries

Each fact carries a ProvSQL provenance token; an AFTER trigger binds the fact's confidence to its token. Queries evaluated under the Viterbi semiring [1, 2] return results whose confidence is the product of their supporting facts' confidences along a derivation (max across alternatives): joining a 0.95 measurement with a 0.70 inference yields 0.665, computed by the engine, not by every application re-implementing (or forgetting) the arithmetic. §4 quantifies what the baselines that skip this get wrong.

3.5 Kind-scoped use and progressive depth

Three views make origin-scoped reading a default: `fact_compliance` (MEASURED + DERIVED), `fact_training_safe` (MEASURED only — the decontamination boundary of §2 as a view rather than a pipeline), and `fact_analytics` (all kinds). Complementing them, `expand(entity, depth, min_conf)` widens one query stepwise — depth 0: measured only; 1: + inferred; 2: + derived — so a caller chooses coverage against certainty explicitly and sees the confidence cost of each widening.

3.6 Bitemporal reads

`as_of_valid(entity, attr, t)` answers "what was true at t"; `as_of_believed(entity, attr, t, s)` answers "what did we believe at s about time t" — the shape of question that point-in-time regulatory reporting requires [17] and, combined with the audit relation, the shape a lender needs to defend a past decision.

4 EVALUATION

Setup. PostgreSQL 17.10 and ProvSQL 1.10.0, native ARM64 (Apple silicon), no emulation; seed 42; adversarial suite of 100 write payloads (70 trust-violating, 30 legal) executed 10 times; dataset for loaded runs is 11,637 Synthea [18] synthetic patients yielding 615,658 typed facts. Four systems: `kndb`; `pg_naive` (plain schema); `py_guards` (application-layer checks in Python, 41 LOC); `pg_handrolled` (a steelman: hand-written triggers reproducing the typing and lattice rules, 124 LOC, no confidence propagation). KNDB's engine totals 508 LOC, of which 214 are guard logic.

Enforcement. Table 1 shows the headline: the plain schema accepts every violation; carefully written application guards catch fewer than half; both engine-resident approaches catch all of them. We emphasize the steelman's 70/70 rather than hiding it. What separates KNDB is reuse, not enforcement power: KNDB defines the guarantee once at the database layer where it applies to every writer of the shared store, whereas the steelman's triggers are a per-application, per-table implementation. Similarly, the py_guards 30/70 does not claim application-layer enforcement cannot work; it shows what such enforcement typically looks like when re-implemented per application at 41 LOC of scope.

System	caught/70	p50 (μs)	p95 (μs)	rps
kndb	70	84.9	183.4	9,958
pg_handrolled	70	69.8	163.2	12,036
py_guards	30	—	—	—
pg_naive	0	—	—	—

Table 1: Adversarial suite, empty database, native hardware.

Confidence correctness. Over 100 inner-join chains with product-of-confidences ground truth, kndb and the steelman-with-propagation-disabled diverge sharply: kndb matches exactly on 100/100 (zero drift); pg_naive and py_guards, which carry no confidence, drift by 0.210 mean and 0.426 max absolute error when their flat results are compared against the true composed confidence. The point is not that the baselines compute confidence badly; it is that they do not compute it at all, and downstream consumers cannot tell.

The cost, honestly. Engine-resident trust is not free, and the cost has structure. Per-write cost in kndb is $O(1)$ trigger work plus an $O(\log|\text{fact}|)$ term from ProvSQL's token-to-probability map, which grows with the table; the steelman, which does not propagate confidence, is size-invariant. Empirically, across our two benchmark generations the pattern is consistent though its magnitude varies: in the v1 run kndb dropped from 13,150 rps (empty) to 5,598 rps at 565k rows (v1 dataset) (−57.4%) while the steelman was flat; in the current v2 run kndb drops from 9,958 to 7,574 (−23.9%) at 615k rows while the steelman moves −3.1%. We therefore make the claim qualitatively: size-dependent write cost exists in kndb because provenance bookkeeping grows with the table; a system that skips propagation avoids that cost by not providing the guarantee. Separately, the v2 precedence lattice costs roughly 3% throughput over the old confidence-wins trigger on like-for-like runs, with part of the measured spread attributable to runner noise (the steelman, with no code change, moved −19.3% between the same runs).

Progressive depth on real data. On the Synthea corpus, depth 0 returns 544,349 live rows (measured labs on five LOINC codes); depth 1 adds 4,999 inferred predictions (mean confidence 0.70); depth 2 adds 16,239 derived 90-day aggregates: 565,587 Synthea rows total, with recall monotonically up and average confidence monotonically down — the coverage-versus-certainty trade made explicit and queryable.

Reproducibility. `make smoke` runs an eleven-stage end-to-end self-validation (write acceptance, R5 rejection, lattice precedence including the high-confidence-inference-versus-lower-confidence-measurement case, Viterbi arithmetic to within 10^{-3}, bitemporal reads, depth monotonicity, view exclusions, audit reasons) in 2 seconds warm; a fresh clone reaches a passing suite in 9.6 seconds. All numbers in this paper carry paths to their raw CSV/JSON in the repository.

5 RELATED WORK

Provenance and probabilistic databases. Semiring provenance [1] and ProvSQL [2] supply our propagation machinery; probabilistic systems such as Trio [4] and MayBMS [5] pioneered uncertainty as data. Widiaatmaja et al. [3] extend ProvSQL with update provenance and demonstrate a temporal database over a union-of-intervals m-semiring, sharing our provenance substrate; that contribution is a demonstration of temporal and update provenance, while KNDB adds write-time epistemic-kind enforcement, kind-ranked conflict resolution, and use-scoping on top. KNDB's claim is not the mathematics but its enforcement position: kind and confidence checked at write time and consulted at conflict- and use-time in a general-purpose store, rather than available to queries that opt in.

Agent-memory systems. A recent line of work brings temporal and contradiction machinery to LLM-agent memory: TOKI [6] types contradiction-resolution heuristics as bitemporal operators with provenance-preserving audit rows and soundness theorems; related systems add typed memory representations and governed retrieval [7, 8]. We adopt the same classical substrates (bitemporal models, provenance) but differ in scope and mechanism: these systems manage one agent's belief store, whereas KNDB proposes origin as an engine property of a shared, general-purpose database with many writers; and none of them types the data itself epistemically, ranks conflicts by that kind, or scopes use by it. Romanchuk and Bondar [9] formalize *semantic laundering* — the way generated propositions gain unearned epistemic status when they cross trusted boundaries — as an architectural Gettier problem. KNDB is a concrete database-layer instantiation of that guardrail: the write-time kind check is precisely the boundary at which such laundering can be refused.

MDM and data standards. Master-data survivorship with source trust scores [20] is the closest industrial practice: it ranks sources at merge time, in a hub, by configuration. KNDB ranks the epistemic kind of each fact, at write time, in the engine, with the loser preserved and the reason recorded. FHIR's observation status and W3C PROV [19] demonstrate that standards bodies recognize the measured-versus-derived distinction; they specify how to represent it, not an engine that enforces it. Active databases and production-rule systems [21] contributed specificity-based conflict resolution; our lattice composes that classical idea with an epistemic axis it did not have.

Positioning. Every mechanism in KNDB has ancestry. Our contribution is the position that origin should be a database-layer property governing writes, conflicts, and use in a general-purpose store, together with evidence that the composition is buildable in 508 lines on stock infrastructure, with measurable enforcement value and measured, structured cost.

6 LIMITATIONS AND FUTURE WORK

KNDB governs origin, not value correctness: a correctly-labeled measured write carrying a wrong number passes; that is the province of CHECK constraints and validation, which are complementary. Preliminary concurrency testing indicates KNDB's enforcement guarantees hold under single-writer workloads or PostgreSQL SERIALIZABLE isolation; full characterization across isolation levels and contention regimes is in progress. The audit trail for rejected (as opposed to evicted) writes further requires an autonomous-transaction path we have scoped out. The specificity axis is exercised by tests but its calibration across real organizations is open. ProvSQL's propagation cost motivates either batched `set_prob` maintenance or propagation-on-read designs. An epistemically-aware validation layer — stricter plausibility checks for inferred values than measured ones — is a natural extension we deliberately did not build. Finally, the deepest open question is standardization: if origin belongs in the engine, it ultimately belongs in SQL.

7 CONCLUSION

Databases learned long ago that guarantees which depend on every application behaving are not guarantees; that is why types, constraints, and referential integrity live in the engine. We have argued that a fact's epistemic kind now belongs on that list: enrichment pipelines, model outputs, and agents write guesses beside measurements at scale, and the cost of conflating them is documented in survivorship failures, unexplainable decisions, and contaminated training corpora. KNDB shows the primitive set is small, buildable on stock PostgreSQL in 500 lines, exact where it claims exactness, and honest about what it costs. The mechanisms are ordinary. The position we ask the community to debate is that the engine should know how a value came to exist.

ACKNOWLEDGMENTS

Portions of this manuscript were prepared with the assistance of AI tools for language editing and formatting. All research content, ideas, experimental design, code, and citations are the authors' work, and the authors take full responsibility for the final content in accordance with the ACM Policy on Authorship.

REFERENCES

[1] T. Green, G. Karvounarakis, and V. Tannen. 2007. Provenance semirings. In *Proceedings of PODS 2007*.

[2] P. Senellart, L. Jachiet, S. Maniu, and Y. Ramusat. 2018. ProvSQL: provenance and probability management in PostgreSQL. *Proceedings of the VLDB Endowment*, 11(12).

[3] A. A. Widiaatmaja, B. Djeffal, A. Dandekar, and P. Senellart. 2025. Demonstration of ProvSQL update provenance through temporal databases. In *Proceedings of ProvenanceWeek (PW) 2025*, Berlin.

[4] J. Widom. 2005. Trio: a system for integrated management of data, accuracy, and lineage. In *Proceedings of CIDR 2005*.

[5] C. Koch et al. 2009. MayBMS: a probabilistic database management system. In *Proceedings of ACM SIGMOD 2009*.

[6] Z. Wang. 2026. TOKI: a bitemporal operator algebra for contradiction resolution in LLM-agent persistent memory. *arXiv preprint arXiv:2606.06240*.

[7] Z. Jin, B. Wang, J. Li, R. Xu, and M. Zhang. 2026. Mitigating provenance-role collapse in long-term agents via typed memory representation. *arXiv preprint arXiv:2605.25869*.

[8] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit. 2026. Governed shared memory for multi-agent LLM systems. *arXiv preprint arXiv:2606.24535*.

[9] O. Romanchuk and R. Bondar. 2026. Semantic laundering in AI agent architectures: why tool boundaries do not confer epistemic warrant. *arXiv preprint arXiv:2601.08333*.

[10] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal. 2024. AI models collapse when trained on recursively generated data. *Nature*, 631.

[11] P. Laban, T. Schnabel, and J. Neville. 2026. LLMs corrupt your documents when you delegate. Microsoft Research / arXiv preprint, April 2026.

[12] M. Zawalski, M. Boubdir, K. Bałazy, B. Nushi, and P. Ribalta. 2025. CoDeC: detecting data contamination in LLMs via in-context learning. *arXiv preprint arXiv:2510.27055*.

[13] Consumer Financial Protection Bureau. 2023. Circular 2023-03: adverse action notification requirements. September 2023.

[14] Consumer Financial Protection Bureau. 2025. Order imposing $15M civil penalty on Equifax. January 2025.

[15] The Data Governor. 2026. Is your MDM "good enough"? The practitioner's checklist. April 2026. https://thedatagovernor.com/mdm-good-enough-checklist/

[16] RevBlack. 2026. CRM data quality problems: why your CRM is wrong. April 2026. https://www.revblack.com/articles/data-quality-nightmares-why-your-crm-is-lying-to-you

[17] Basel Committee on Banking Supervision. 2013. BCBS 239: principles for effective risk data aggregation and risk reporting.

[18] J. Walonoski et al. 2018. Synthea: an approach, method, and software for generating synthetic patients. *Journal of the American Medical Informatics Association*, 25(3).

[19] World Wide Web Consortium. 2013. PROV-O: the PROV ontology. W3C Recommendation.

[20] Informatica. *Trust settings and validation rules. Multidomain MDM 10.5 Configuration Guide.* https://docs.informatica.com/master-data-management/multidomain-mdm/10-5/configuration-guide/

[21] U. Dayal et al. 1988. The HiPAC project: combining active databases and timing constraints. *SIGMOD Record*, 17(1).